

Make at the center of your pipelines

Make (Integromat) is Expensive. WPipe is Efficient. 🌱

The context

Paying per execution? That's old school. WPipe allows you to scale without scaling your costs.

Why engineers shift to wpipe

- Unlimited Executions: It's your code, on your hardware.
- Efficiency: Optimized SQLite WAL mode means minimal disk I/O and near-zero idle RAM.
- Resilience: If your local machine reboots, WPipe resumes. Make just "fails."

Code-first

Pipelines are Python: versionable, testable, reviewable in Git.

Checkpoints

State persisted to SQLite WAL at every step; resume, don't restart.

Independently lean

<50MB RAM. No broker, no daemon, no metadata server.

What it looks like

```
from wpipe import Pipeline, step
from wpipe import Parallel

@step(name="branch_a")
def branch_a(data): return {"a": data["x"]}

@step(name="branch_b")
def branch_b(data): return {"b": data["y"]}

pipe = Pipeline(pipeline_name="make_to_wpipe")
pipe.set_steps([Parallel(steps=[branch_a, branch_b])])
```

At a glance

Aspect	Legacy / Make	wpipe
Definition	Config/DSL or canvas	Python code
State	Scattered / none	SQLite WAL checkpoints
Observability	Partial	Tracker SQL + dashboard
Footprint	Heavy	<50MB RAM

Under the hood

The flow behind the post, as wpipe renders it from code.



+117k downloads

A community of efficiency-first developers.

<50MB RAM

Fits containers, VMs, and Raspberry Pi-class hardware.

SQLite WAL

Industrial durability without a database to administer.

Next step

wpipe is a Python library — try it in your next pipeline:

```
pip install wpipe
```

Steps stay functions you already understand. Docs, diagrams, and dashboards follow. Explore **Make** workflows at wpipe.readthedocs.io.

wpipe · Python-native orchestration with SQLite WAL checkpoints, retries, timeouts, parallelism, and self-documenting flows. +117k downloads.